TRANSURBAN — DS em 5 minutos

José, Bruno e Kauan • Apresentação do backend • Sem slides

José faz todos os cliques e as explicações técnicas. Bruno e Kauan têm falas curtas. Duração aproximada de 5 minutos, incluindo os testes. Leiam somente os trechos FALA. NA TELA orienta o operador. Ensaiem uma vez com cronômetro.

ANTES DE COMEÇAR: deixe a API iniciada, o terminal aberto e http://127.0.0.1:8000/docs no navegador do mesmo computador. Confira GET /health antes da fala: status ok e banco sqlite_local. Não abra .env no telão. Deixe database.py, models.py, schemas.py e main.py preparados no VS Code.

0:00–0:35 — José: abertura

FALA: “Bom dia, pessoal! Eu sou o José e, junto com o Bruno e o Kauan, vou apresentar o backend do TRANSURBAN.

FALA: O backend é a parte que recebe os pedidos, verifica as informações e trabalha com o banco de dados. Vamos mostrar isso pela API: cadastrar uma cidade, ligar uma linha de ônibus a ela, consultar os registros e testar entradas inválidas.”

0:35–1:00 — Bruno: objetivo do sistema

FALA: “Nosso sistema organiza cidades e linhas de ônibus. Primeiro cadastramos a cidade e depois uma linha ligada a ela. Isso permite consultar uma linha e saber a qual cidade ela pertence. Essa organização ajuda a manter as informações relacionadas no banco.”

1:00–1:25 — Kauan: pedidos e respostas

FALA: “Para enviar um cadastro, usamos POST. Para consultar os dados que já estão salvos, usamos GET. A API responde em JSON, que organiza as informações em campos e valores. Agora o José vai mostrar como isso funciona na prática.”

1:25–2:00 — José: arquivos do backend

NA TELA: mostre brevemente os quatro arquivos no VS Code. Não leia o código inteiro. Depois volte ao Swagger.

FALA: “Separamos o código por responsabilidade. O database.py configura a conexão com o banco. O models.py define as tabelas e seus relacionamentos. O schemas.py valida os dados recebidos. O main.py disponibiliza as rotas da API.

FALA: Usamos Python com FastAPI para as rotas, SQLAlchemy para trabalhar com o banco e Pydantic para validar as entradas. Na apresentação, o banco é SQLite local.”

2:00–3:20 — José: cadastro e relacionamento

NA TELA: POST /cidades → Try it out. Cole o JSON e clique Execute:

```
{"nome":"Sarandi"}
```

NA TELA: espere 201 e anote o id_cidade retornado. Não suponha que seja 1.

FALA: “Cadastramos Sarandi e recebemos 201, indicando que o registro foi criado. O id_cidade é a chave primária: identifica esse registro de forma única.”

NA TELA: em POST /linhas, substitua o 1 pelo id_cidade realmente devolvido. Depois clique Execute:

```
{
  "codigo": "DEMO-20",
  "nome": "Linha da apresentação",
  "id_cidade": 1
}
```

NA TELA: espere 201. Aponte id_cidade e cidade na resposta.

FALA: “Agora cadastramos uma linha de demonstração. O id_cidade é a chave estrangeira: faz a ligação entre a linha e a cidade cadastrada.

FALA: Neste recorte do sistema, uma cidade pode ter várias linhas cadastradas, e cada linha aponta para uma cidade existente. A API verifica essa existência antes de salvar.”

3:20–3:55 — José: consultar o que foi salvo

NA TELA: execute GET /cidades e GET /linhas. Cada consulta deve retornar 200. Localize o cadastro desta execução; a lista pode ter registros de ensaios anteriores. Confira também que o ID 999999 não está entre as cidades.

FALA: “Agora usamos GET para consultar os registros. O código 200 indica que a consulta foi atendida. A resposta mostra a linha junto do nome da cidade, facilitando a compreensão.

FALA: Os dados foram salvos no banco local. Eles não ficam apenas no formulário que usamos para cadastrar.”

3:55–4:45 — José: testar entradas inválidas

FALA: “Agora vamos informar uma cidade inexistente, de propósito, para testar a proteção da API.”

NA TELA: se 999999 foi confirmado como ausente, envie em POST /linhas:

```
{
  "codigo": "TESTE-ERRO",
  "nome": "Tentativa inválida",
  "id_cidade": 999999
}
```

NA TELA: esperado 404, com detail: Cidade não encontrada. Execute GET /linhas novamente e confira que essa tentativa não criou um novo registro. Se houver TESTE-ERRO de outro ensaio, compare os IDs antes e depois.

FALA: “A API recusou o cadastro e retornou 404: cidade não encontrada. A tentativa não foi salva.”

NA TELA: em POST /cidades, envie um nome formado só por espaços:

```
{"nome":"   "}
```

NA TELA: esperado 422. Não leia toda a mensagem técnica.

FALA: “Também testamos um nome com apenas espaços. A validação rejeitou essa entrada com 422. Esses resultados são esperados e mostram que o sistema verifica os dados antes de aceitar o cadastro.”

4:45–5:00 — José: encerramento

FALA: “Mostramos a organização do backend, o cadastro, a consulta, a ligação entre tabelas e a validação das entradas. Também usamos testes automatizados e versionamos o código no GitHub. Obrigado pela atenção!”

Apoio rápido — não faz parte dos 5 minutos

Se perguntarem	Resposta
O que é PK?	Chave primária: identifica um registro.
O que é FK?	Chave estrangeira: liga o registro a outra tabela.
Por que 404 e 422 apareceram?	Foram erros provocados de propósito: cidade inexistente e nome inválido.
Qual o banco da apresentação?	SQLite local, informado por GET /health. Não anunciar conexão Supabase sem comprovação.
Se reiniciar, os dados continuam?	O inicializador usa o arquivo apresentacao_local.db. Não o apague. Para demonstrar, reinicie o servidor com a mesma configuração e consulte o mesmo id_linha.
Cidade repetida é rejeitada?	Não neste modelo SQLite atual: nomes não têm UNIQUE. Use sempre o ID realmente retornado.

O código DEMO-20 é um exemplo de cadastro, não uma linha operacional verificada. Se o ID 999999 existir, escolha outro ID positivo confirmado como ausente. Se um resultado for diferente do esperado, não anuncie sucesso antes de conferir. A duração depende do ensaio e da velocidade dos cliques.